Supplementary Material for: Are Long-Running Agents Making Progress? A Longitudinal Study of Artifact Progress, Repeated Mutation, and Continuity

Anonymous submission

Abstract

Long-running agents can modify a workspace for days across many session boundaries. When they stop, action counts, commits, and final artifacts do not reveal whether activity accumulated into durable and validated progress or repeatedly overwrote, abandoned, and rediscovered the same work. Existing trajectory studies characterize actions within benchmark attempts, while studies of long-lived agents focus on task outcomes or internal memory degradation. We instead make the persistent workspace the longitudinal unit of analysis. We introduce a source-linked projection that connects native Claude, Codex, and Gemini actions to stable artifact identities, lifecycle effects, event time, and session boundaries without introducing semantic labels or aligning the timeline to Git commits. We use it to study six longitudinal questions over six real software and autonomous-research projects—introduced-artifact persistence, later reuse, validation association, repeated mutation, workspace-activity migration, and cross-session continuity—plus a seventh question about tool-call workload and conservative optimization-opportunity bounds. Under the admitted local projection, across 551 native root sessions and 181,303 Tool actions, 89.3–97.1% of eligible mutations are later revisited, yet action volume only weakly orders the cross-project reuse relation ($\rho = 0.20$). Repeat-observed episodes account for 74.6–90.8% of mutation episodes, with 42.0–86.7% of episodes concentrated in the most-mutated 10% of identities. Source coverage is itself consequential: recognized successful validation appears in all six cases after projection repair, and named Skills show no supported repeatable separation in the only eligible comparison. In 320 stratum-specific public task/topic selections, path-local transitions and 2–3-intervening-call module returns recur, but persistent artifact lineage and cross-session re-grounding remain unobservable. Native overlap already consumes 86.0% of logical-parallel edges, while an actionable next-read prefetcher is only 21.75% precise. Finally, a source-direct experiment moves from 32/60 to 60/60 artifact-linked plus cross-session facts after repair on its corrected repair corpus; a preregistered 70-session held-out test instead obtains 54/58 and attempted-edge F1 0.9950. These are corpus-specific conformance results, not a general exact-fact capability claim.

Introduction

An increasingly common workflow gives an agent a broad idea, leaves it to work inside one repository for two or three days, and inspects the result later. The returned workspace may contain hundreds of tool calls, dozens of commits, and substantial code, prose, tests, and experimental output. This volume looks productive but does not answer the question the user actually has: *did the work converge into durable, validated progress?*

Final state and commit history are necessary but lossy views of that question. They omit reads, failed validations, transient artifacts, overwritten attempts, and the cost of re-establishing context after a session boundary. Aggregate counts retain activity volume but erase which artifacts actions concerned and whether later work reused or replaced them. A raw log retains detail, yet its session-local linear order does not directly expose a workspace lineage that continues after one context disappears. Context management can rapidly weaken earlier plans (Mehta and Datta 2026), and reliability can change over long chains of sessions (Zhu et al. 2026).

Fine-grained trajectory analysis is not new. Prior work mines action sequences and repeated behaviors in software-engineering agents (Bouzenia and Pradel 2025), measures validation and context-gathering behavior under task and model confounds (Mehtiyev and Assunção 2026), and shows that agents may reach a correct intermediate state and later overwrite it (Kim et al. 2026). ProcGrep represents trajectories as action procedures and supports deterministic pattern queries and behavioral fingerprints (Oderinwale 2026). Persistent file workspaces and multi-session tasks are also established system mechanisms and settings (Zhou et al. 2026). These results rule out generic claims about first trajectory analysis, edit loops, validation gaps, fingerprints, or persistent workspaces.

The remaining gap is longitudinal and artifact centered. A benchmark attempt usually ends when one patch is scored; a persistent project continues while code, papers, datasets, results, and documentation acquire histories of their own. We ask how action volume relates to three observable dimensions: *introduced-artifact persistence*, whether an observably introduced artifact persists; *reuse*, whether later work returns to it; and *validation association*, whether an adapter-recognized successful validation precedes the artifact's next mutation or deletion. We keep these dimensions separate instead of com-

bining them with arbitrary weights. Their conjunction is useful shorthand for durable, reused, and validation-associated artifact progress, not a scalar quality score.

We build this study directly on native Agent records. The existing `agent-session` adapters parse Claude, Codex, and Gemini. A thin repository projection retains all admitted Tool actions, resolves qualified file effects, attempts to preserve explicit rename lineage and session identity, and orders events by Agent action time. Git contributes final tracked state and optional content-survival evidence, but commits never define the time axis. The same facts drive both quantitative analyses and the auxiliary AGENT NEBULA replay; visual layout is not a statistical measurement. We independently test the projection rather than treating its output as truth.

This paper makes three contributions:

1. a projection-conditioned longitudinal characterization of how Agent activity, artifact persistence, reuse, validation, repeated mutation, workspace activity, session continuity, and tool-call workload bounds interact across six real coding and autonomous-research projects;
2. a deterministic, source-linked measurement protocol for stable artifact identity and cross-session lineage, together with a source-direct conformance benchmark on which the frozen implementation failed (32/60 artifact-linked and cross-session facts, 2026-07-23) and the repaired revision now answers 60/60 as repair-corpus regression evidence, followed by a separate preregistered root-disjoint held-out result of 54/58 rather than a general exact-fact capability claim; and
3. a reproducible coverage discipline that separates measured results, coverage-only evidence, implementation disagreement, and N/A conditions instead of treating missing or projected trajectory evidence as observed truth.

Problem Setting

Persistent Work Rather Than Isolated Sessions

Let a workspace W persist while sessions $S = (s_1, \ldots, s_m)$ execute native Tool actions $A = (a_1, \ldots, a_n)$. Each action may read, create, mutate, rename, or delete zero or more artifacts. Sessions partition model context; they do not reset W or terminate an artifact's history. Our unit of analysis is one repository lineage over the complete observation interval, with project-level cases kept separate.

Action time is ordered by native timestamp and stable source identifier. A commit can occur between actions and can anchor a final-state query, but it does not replace the action order. This distinction is important because an agent can explore, fail, create temporary files, and revert changes before any commit.

Observable Progress Without Semantic Gold

We do not infer intent, failure cause, reflection, or waste from file motion. Instead, we measure source-verifiable process facts:

- **Introduced-artifact persistence**: final existence or tracked state for artifacts observably introduced during the trace; existing-file writes have unknown content durability without independent evidence;
- **Reuse**: a later source-linked access to the same artifact lineage and its event, time, and session distance; and
- **Validation association**: distance from mutation to the next adapter-recognized successful test, check, or build action before that artifact's next mutation or deletion.

Recognized validation proves only that the adapter observed a successful command, not that it covered a particular mutation; unrecognized commands and unknown statuses remain coverage gaps. A file that is never revisited is an observed orphaned artifact, not automatically useless. Repeated tests or edits are process patterns, not automatically waste. Skill and harness analyses therefore report temporal association, never causality, in the observational corpus.

Figure 1 makes the measurement relationships explicit. The study follows Agent action time through session boundaries, keeps artifact histories distinct, and measures survival, later reuse, validation distance, and observable re-grounding. The figure is a definition schematic rather than an empirical result; the corresponding result plots are generated from the six-project source rows in Section .

Workspace-Centered Measurement

Source Selection and Projection

We admit a complete native session when its project directory, working directory or worktree root, or Git remote identifies the target repository. All Tool actions in an admitted session remain on the timeline, including searches, shell commands, failed validations, and calls without a resolved file path. A global path scan is reported separately because matching only repository-related lines loses the surrounding no-file actions.

For each admitted action, the projection retains

$$a_i = (t_i, id_i, s_i, v_i, u_i, e_i, q_i), \tag{1}$$

where t_i is native time, id_i the stable source ID, s_i the session, v_i the vendor, u_i the Tool/category, e_i the adapter-derived command effect, and q_i the reported status. It attaches zero or more effects

$$\begin{aligned} E_i &= \{(w, f, o, f')\}, \\ o &\in \{\mathsf{read}, \mathsf{write}, \mathsf{create}, \\ &\qquad \mathsf{rename}, \mathsf{delete}\}. \end{aligned} \tag{2}$$

Here w is a hashed worktree identifier, f is a repository-relative artifact, and f' is populated only by an observed rename. Failed calls remain actions but emit no confirmed-success effect. Directory arguments are weak scope evidence rather than fabricated reads of every descendant. Unknown paths, effects, and status remain explicit coverage gaps.

Artifact and Validation Derivations

Artifact identity is keyed by hashed worktree ID and relative path. Tool-level workdir takes precedence over session cwd. An explicit same-worktree rename preserves identity and its birth state; only a confirmed-success create makes an identity

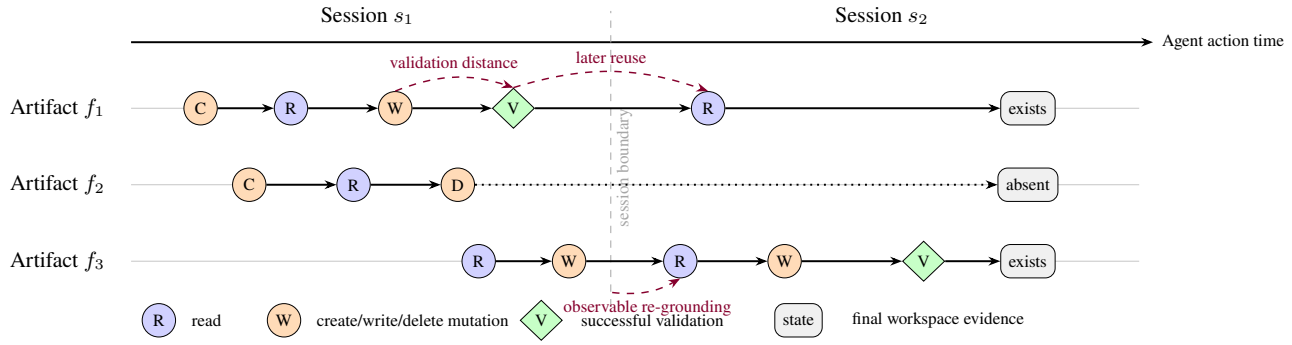


Figure 1: Measurement schematic for persistent workspace evolution. Circles are source-linked reads (R) and mutations (C/W/D); diamonds are successful validations (V); terminal boxes are independently observed final workspace state. Artifact f_1 is mutated, validated, reused after a session boundary, and survives; f_2 is created and later deleted; f_3 illustrates observable re-grounding before continued mutation. The study reports these dimensions separately and never treats the schematic as empirical data.

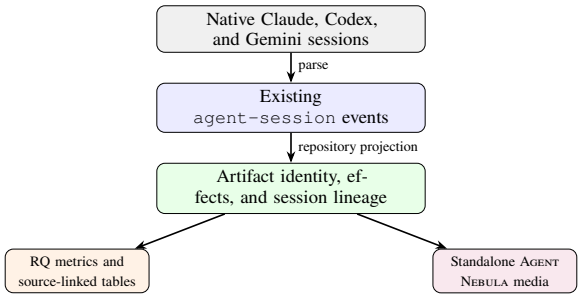


Figure 2: One source projection supports measurement and visualization. The empirical analyses consume structured events, not visual coordinates.

eligible as an introduction inside the observation. Unknown-status creates retain a separate birth state. Delete followed by create begins a new identity. For each confirmed non-delete mutation, we locate later reuse and adapter-recognized successful validation before the same artifact's next mutation/delete. Supersession is a competing outcome and only observation end is right-censored. Final existence and tracked state are queried in the corresponding worktree.

We report complete survival and distance curves or declared sensitivity ranges; no fixed 24-event attention window enters the research measurement. Visual attention decay and force layout remain presentation-only choices.

Matched-Comparison Prerequisites

A valid tool comparison would stratify questions into action-only, artifact-linked, cross-session, and final-state facts. Every answer would cite source IDs or abstain, and no method under comparison could label its own answer. Final State and Counts would be lower-information controls, ProcGrep the strongest action-only procedure baseline, and a bounded raw-log model a same-source reconstruction condition.

Step 0004 satisfies these prerequisites for the deterministic conditions: it freezes hashed native prefixes and cutoff state, independently reconstructs the oracle, and executes the 480-row Final State, Counts, ProcGrep, and Trajectory matrix. The bounded Raw matrix materializes the remaining 360 denominator rows, so the planned 840-row denominator is complete under the registered terminal-status rules. Section reports the fixed-reader, fixed-corpus comparison.

Implementation

The implementation is a Rust extension of the existing `agentvis` repository projection. It reuses `agent-session` for native parsing, affiliates sessions with all repository worktrees, normalizes paths, and emits ordinary source-linked JSON and CSV rows. The analysis computes project-level coverage, artifact and mutation rows, and RQ summaries without a frontend, database, generated semantic layer, or additional runtime dependency. It reports all admitted Tool actions but uses only actions whose Tool-level workdir or session cwd resolves to a retained worktree for the primary activity–progress comparison.

The product path continues to export one standalone HTML, SVG, PNG, GIF, or MP4 per visualization. Layout is computed once and media outputs share the same frame stream. These outputs provide case evidence navigation and reproducible examples; the empirical measurements are recomputed from structured rows.

Empirical Study

Research Questions

RQ1—Artifact consolidation and revival. How do introduced and repeatedly modified artifacts persist, receive later reuse, concentrate work, become dormant, and return across Agent action time?

RQ2—Validation response. How do successful and failed validations interleave with mutation bursts, backlog, and subsequent workspace changes?

RQ3—Workspace focus evolution. How are path-resolved actions allocated among artifact types, and how do artifact/module hotspots migrate, cool, and receive later returns?

Case	Visibility and dominant work type	Active	Days
A	public; systems software/research	2	136.17
B	public; systems software/research	3	56.11
C	public; tutorial/code/documentation	2	191.76
D	public; content/documentation/software	2	134.35
E	private; paper/experiment/code	1	0.13
F	private; skills/prompts/tests/documents	1	32.40

Table 1: Anonymous fixed-case profiles. Span is from the earliest to latest admitted native-root session start. Each project remains one case; its sessions are not treated as independent projects.

RQ4—Cross-session continuity. What re-grounding and prior-artifact/module continuity is observable between adjacent non-overlapping native-root session components?

RQ5—Skill and instruction footprints. What workspace-action composition is explicitly attributed to named Skills across independent native root sessions, is it more repeatable than matched different-Skill activity, and what immediate activity follows explicit instruction-file access?

RQ6—External boundary. Which compatible within-attempt relations replicate in public coding and scientific-process traces, and which longitudinal claims remain N/A without persistent cross-session workspace lineage?

RQ7—Tool-call workload and optimization bounds. What composition, repetition, dependency, and waiting structure does the long-running tool-call workload exhibit, and what conservative bounds does that structure place on prefetch, concurrency, speculation, caching, and event-driven control?

Separate tool question—Measurement capability. Given the same frozen native evidence and an independent source-explicit oracle, which action-only, artifact-linked, cross-session, and final-state facts can Final State, Counts, ProcGrep, bounded raw-log model analysis, and artifact-linked trajectories answer correctly, and at what evidence and inference cost?

Cases and Inclusion

We fix six author-associated local projects before computing process rates. Four are public MIT-licensed repositories from one open-source community; two are private repositories. At a 2026-07-26 snapshot, the four public cases sum to 5,053 repository stars, while all 144 public repositories in their community sum to 9,924 stars. The latter is an aggregate of repository star counts, not a count of unique users or a GitHub organization-level metric. Across the public cases, two or three currently authorized `admin`/`maintain` accounts have a GitHub-linked commit in the case's observation window; each private case has one. We call these accounts verified active maintainers. The admitted session starts span 191.8 days (about 6.3 months) overall.

The main corpus includes all repository-direct Claude, Codex, and Gemini sessions and retains actions with no file effect. We report source coverage by project, vendor, session, effect, status, and path presence. Global path-only matches form a separate sensitivity analysis. The corpus contains 551 admitted native root sessions (2,049 source transcript files before subagent and continuation deduplication) and 181,303 Tool actions; 551 roots and 176,288 actions resolve to retained worktrees. It yields 5,746 observed artifact identities and 13,906 confirmed mutation rows. All six projects pass the longitudinal gate, although source coverage for individual measurements varies substantially.

Analysis Protocol

Each project is first analyzed as a complete case. Cross-case summaries state their project weighting and do not treat actions as independent samples. Primary outputs are effect sizes, distributions, cumulative/survival curves, and uncertainty. Bootstrap or permutation uses session or project blocks only when exchangeability is defensible. Missing source fields remain coverage results rather than imputed events.

Artifact type is an inspectable path/extension classification with an unknown class. Top-level directories provide the default module unit. Model-derived motifs, embeddings, clusters, or summaries are exploratory secondary analyses and must resolve to source IDs and deterministic metrics.

RQ1: Artifact Consolidation and Revival

RQ1 reports action and mutation volume, introduced-artifact persistence, later reuse distance, validation-before-supersession distance, competing outcomes, and the conjunction for eligible observation-born introduction episodes. Existing-file writes have unknown content durability unless native diffs, snapshots, or Git line evidence establish it independently.

Figure 3 separates the three dimensions. All six projects support later-reuse analysis: 89.3–97.1% of eligible mutations receive a later observed access, with delete before reuse treated as a competing outcome. After projection repair, all six projects also expose an eligible confirmed create and an adapter-recognized successful validation, so the preregistered four-project gate admits cross-case interpretation of persistence and validation (it stopped at 3/6 coverage before repair). The six observable persistence proportions are 973/1042, 28/239, 18/30, 8/18, 18/18, and 1/1, and the corresponding observed validation-before-supersession proportions are 2450/6112, 1210/5604, 141/694, 47/282, 9/196, and 33/247.

Figure 4 tests whether raw action volume is an adequate proxy for these outcomes. Reuse covers all six cases, and its descriptive Spearman rank association with worktree-attributed Tool action volume is $\rho = 0.20$ across six cases. This weak six-case rank contrast does not estimate a population effect, but it shows that action count alone only weakly orders the observed reuse relation in this corpus. RQ1 thus establishes measurable activity–progress separation for reuse while, after repair, also admitting cross-case persistence and validation description.

RQ1 Extension: Dormant-to-Revived Transitions

We replay confirmed, non-scope artifact touches in native action order, collapsing compound effects from one action on

one identity. An identity becomes dormant when consecutive touches are separated by strictly more than 100 intervening Tool actions or strictly more than 24 elapsed hours; its first subsequent touch is a revival. This observed lifecycle relation is distinct from later mutation reuse and is not a progress score.

Table 2 reports both thresholds. Under the action-gap definition, revived identities account for 8.3–48.3% of all observed artifacts and 42.9–75.1% of artifacts with at least two touches. The time-gap definition yields 0.0–40.0% and 0.0–60.0%, respectively. The two definitions identify 11,271 and 2,285 revival transitions; only 348 and 41 revivals, respectively, are mutations, so revival denotes renewed access rather than necessarily renewed editing.

RQ2: Validation Response

RQ2 measures mutation bursts, recognized successful and failed validation cadence, and worktree-local mutation accumulation between recognized successes. It never calls a successful command proof that every preceding change was covered or correct. Figure 5 shows the exact action-order trajectories and complete inter-success intervals. After projection repair, all six projects expose a recognized successful validation (3/6 before repair), so the predeclared four-project gate admits a cross-case answer. Recognized success covers 6/6 projects, 5/6 form complete inter-success intervals, and 4/6 contain a recognized failure. Across the seven eligible worktree lanes in those five projects, the observed zero-mutation fraction among complete intervals is 29.3–86.1%, while their maxima range from 1 to 817 mutations. We do not assign a distribution-family label without a count-model baseline. Outcome-conditioned response has no consistent cross-case direction: the three higher-event-volume success/failure cases usually validate again before mutating after failure, whereas the sparse three-failure case mutates first. This is evidence about adapter-recognized cadence, not redundant testing or mutation coverage.

RQ1 Supplement: Repeated-Mutation Structure

RQ1 also treats each artifact's first observed mutation episode separately from later observed episodes. Figure 6 shows that repeat-observed episodes account for 74.6–90.8% of observed mutation episodes across all six qualified cases. Concentration differs substantially: the exact top 10% of mutated identities account for 42.0–86.7% of episodes. These are descriptive properties of the observed trajectories. They do not identify a tail family, convergence, thrashing, defect repair, or wasted work; validation-followed revision and module switching remain open facets of the broader question.

RQ4: Cross-Session Continuity

Native records expose root/subagent identity unevenly across vendors, and many source streams share one native root session or overlap in time. RQ4 therefore keeps shared-root streams in one independent block and forms transitive concurrency components within each worktree lane and compares only adjacent non-overlapping components. Figure 7 reports 121 components and 111 boundaries, including mutation-observed prefix composition and defined artifact/module overlap. Fewer than four projects reach 20 eligible boundaries for every estimator; the preregistered gate stops. A recompute with corrected native-root and subagent identity confirms the stop is data-limited (three of six projects reach 20 boundaries), not an artifact of session identity. The figure is thus a source-coverage and within-case continuity audit, not evidence of session reset cost, resumption, model memory, comprehension, or forgetting.

RQ3: Workspace Focus Evolution

RQ3 analyzes path-resolved activity rather than latent attention or time spent. Figure 8 separates access from mutation and retains both `ok` and `observed` statuses. Status sensitivity is material: for example, Case D's mutation allocation to paper/docs is 39.2% with all resolved statuses and 88.2% with `ok` only. Therefore neither stratum is silently promoted to ground truth.

Figure 9 follows each worktree lane in native call order. Across the six cases, successive path-resolved calls remain on the same artifact 29.3–81.6% of the time, move within the same module 17.5–64.6%, and cross modules 0.9–20.8%. Five cases meet the return-gap gate, with median return gaps of 2–4 intervening calls; the AgentSkill case has only three returns and is N/A. These path-transition and return measurements do not imply duration, internal attention, importance, productivity, entropy, or forgetting; rank-set cooling is analyzed separately below.

RQ3 Extension: Rank Turnover and Cooling

We rank the five most-touched artifacts and top-level modules in 100-action windows, using a 50-action stride as the primary view and non-overlapping 100-action windows as a sensitivity view. Across 3,372 nonempty adjacent primary-window pairs, transition-weighted top-1 changes occur in 49.6% of artifact pairs but 13.6% of module pairs; any top-5 membership change occurs in 91.5% versus 42.0%, and mean membership replacement is 43.5% versus 17.4%. Project medians are similar (50.8% versus 12.5%, 88.7% versus 40.8%, and 42.9% versus 20.6%); the non-overlapping sensitivity preserves the artifact–module ordering while raising top-1 change to 72.1% versus 19.1% and any top-5 change to 97.5% versus 60.4%.

For each origin hot-set membership, endpoint retention asks whether the entity is still hot after a lag, while continuous retention requires membership in every intervening window. In the primary view, membership-weighted artifact endpoint retention falls from 57.0% at one window to 20.4% at eight windows, compared with 84.9% to 62.7% for modules; at eight windows, continuous retention is 4.5% for artifacts and 41.1% for modules. The non-overlapping eight-window sensitivity gives endpoint retention of 17.1% versus 58.5% and continuous retention of 1.9% versus 32.1%. These are rank-set "cooling" curves in Agent action order, not elapsed-time, attention, importance, productivity, or forgetting measurements.

RQ5: Skill and Instruction Footprints

The corrected parser retains exact `Skill` names and arguments, source-native `attributionSkill`, model, native

	>100 intervening Tool actions			>24 elapsed hours		
Project	Revived n/N (%)	Transitions	Gap med./p90	Revived n/N (%)	Transitions	Gap med./p90
Case A	$1{,}271/3{,}267$ (38.9)	6,856	349/4,965.5	$526/3{,}267$ (16.1)	1,086	83.1/409.6
Case B	$662/1{,}809$ (36.6)	3,518	469.5/5,014	$382/1{,}809$ (21.1)	801	81.0/279.2
Case C	$42/170$ (24.7)	73	271/661.2	$21/170$ (12.4)	26	682.9/3,234.8
Case D	$174/360$ (48.3)	805	449/2,339	$144/360$ (40.0)	349	267.8/1,800.4
Case E	$2/24$ (8.3)	3	105/120.2	$0/24$ (0.0)	0	—
Case F	$13/116$ (11.2)	16	167.5/360.5	$18/116$ (15.5)	23	60.2/605.3

Table 2: Per-project dormant-to-revived results under both strict thresholds. Revived identities are reported over all observed identities; one identity may contribute multiple transitions. Gap columns give median/p90 intervening Tool actions on the left and elapsed hours on the right (Hyndman–Fan type-7 percentiles).

root-session identity, source-stream identity, and prompt index. A separate checker directly rereads all 2,063 included native transcript streams and reconciles all 7,304 projected Skill/instruction signal rows and 205,836 adjacent Tool boundaries without a mismatch. The six cases contain 67 explicit Skill invocations and 1,675 source-attributed Skill Tool actions.

Invocation and execution do not always occupy the same transcript stream: only 476 attributed actions are contiguously preceded by a matching same-stream invocation. This is a conservative coverage diagnostic, not an exact join. We therefore aggregate native-attributed actions by project, vendor, model, source role, root session, and Skill instead of inventing per-invocation delegated episode boundaries. Five exact-context Skill strata meet the minimum of three distinct root sessions, but only Case E contains two or more qualified Skills in one exact project/vendor/model/source-role stratum. Within that case, median Jensen–Shannon distance is 0.116 for 9 same-Skill pairs and 0.123 for 10 different-Skill pairs. The median difference is -0.007; the one-sided exact root-block randomization gives $p = 0.750$ over 12 admissible assignments (four distinct statistic values). The result therefore does not support a stable or repeatable Skill-fingerprint claim even inside the sole eligible case.

Reads and mutations of `AGENTS.md`, `CLAUDE.md`, and `SKILL.md` are analyzed separately as focal events. Figure 11 reports their immediate next Tool action before a native prompt-index change. The primary figure uses the 2,822 direct or simple-shell accesses independently recomputed with high confidence; the broader parser output remains a sensitivity table because arbitrary scripts and unexpanded shell globs are not claimed complete. An access is not proof that an instruction was injected, followed, useful, or harmful.

RQ6: External Boundary

The six local projects are selected natural cases and do not estimate a population rate. We therefore sample 64 independent task instances from each of four Open-SWE-Traces harness/model strata and 64 independent IdeaTrail topics (320 stratum-specific selections total; 255 unique Open-SWE task IDs occur across the 256 Open-SWE selections). Selection uses a fixed identifier- hash prefix, and one trajectory is retained per task/topic. The five strata are analyzed separately; a second source parser reconciles all 31,249 Tool calls and 22,113 adjacent path-target transitions without a mismatch.

Figure 12 reports the exact common estimands. Aggregate cross-module movement is 18.0–30.0% across the public strata, compared with 2.1–20.2% in the six path-compatible local anchors. Thus the ordered path locality relation recurs, but public coding attempts move across top-level modules more often than most local long-running cases. Module-return medians are 2–3 strictly intervening calls publicly and 2–4 in qualified local cases. Exploration before mutation is not a confirmatory result because the public harnesses shape that order; recognized successful validation lacks a portable attempt/status pair. Path concentration and staged revision are analogous descriptions, not stable-artifact replication.

User-Originated Artifact Questions

We answer four user-originated questions with fixed path-classification, confirmed-effect, pairing, and stream–prompt–module rules. Pooled values are micro-weighted over the six selected cases and do not turn them into a population sample.

Created-document revisit. Across 1,066 observation-born paper and documentation artifacts, 29.8% had no later confirmed in-scope action and 62.4% were subsequently read. In comparison, 11.3% of 124 created code artifacts had no later confirmed action and 78.2% were subsequently read. The document reread fraction varied from 0% to 72.2% across the five cases with created documents, so the pooled rate does not describe a uniform practice. Because the export does not identify which documents were explicitly required by an instruction, these values describe all created documents rather than a causal effect of documentation requirements.

Source–test order. Conservative source–test pairing yielded 28 mutation episodes, all in one case: 13 unambiguous normalized-basename pairs and 15 same-event module fallbacks. Seven episodes (25.0%) modified source first, none modified the test first, and 21 (75.0%) modified both in the same Tool event. The same-event fallback intentionally avoids inferring order from temporally distant mutations in a broad module. Because five cases had no eligible pairs, this result is a within-case description rather than evidence for a general code-first strategy.

Artifact-type allocation. Confirmed read actions were nearly balanced between paper/document artifacts (43.5%)

and code (43.2%), whereas confirmed writes were concentrated on paper/documents (69.8%) rather than code (16.3%). Including actions with an `observed` rather than definite status preserved these directions but reduced the write contrast to 54.8% versus 32.1%. Two cases had more code than document reads, but all six had more document than code writes in both status views. These quantities count normalized file actions and do not estimate elapsed time, effort, importance, or progress.

Test churn. We observed 116 executable-test mutation episodes and 2,257 source-code episodes, with repeat-episode fractions of 71.6% and 86.0%, respectively. Within 31 test-bearing stream–prompt–module blocks, 16 contained repeated mutation of a test identity, but none of those 16 had zero source-code episodes. Five blocks had no code mutation, yet none repeatedly mutated the same test identity; the only repeat-test block with more test than code episodes contained two versus one. Test episodes had a higher temporal association with a recognized successful validation (61.2% versus 48.7%), which does not imply that the validation exercised or established the quality of those files.

Session Dynamics and Harness-Shaped Work

These analyses retain project–vendor strata and observable proxies rather than assigning latent cognitive states. Among the 233 native roots with at least 30 calls, the five project–vendor strata with at least ten paired sessions all show a positive median late-minus-early resolved-artifact reread change, ranging from 16.7 to 28.9 percentage points. The direction remains positive after excluding roots with an internal gap over eight hours, while failure rate and edit fragmentation do not show a common shift. We therefore describe this as late-session re-grounding or reuse of seen artifacts, not context degradation.

Startup context work has a right tail rather than a uniformly large tax. Across 362 roots with a complete ten-call prefix, the narrow instruction/Git proxy has median 10% and IQR 0–10%; an extended proxy that also includes repository README and predecessor-artifact rereads has median 20%, IQR 10–30%, and p90 60%. None of the sufficiently supported project–vendor strata shows the hypothesized positive relation between predecessor gap and startup proxy. Predecessor rereads may also be useful continuation, so the extended proxy is not labeled waste.

Harness-shaped work is a stratified footprint, not causal overhead. Strict gross, exclusive, and broad proxies account for 6.48%, 5.92%, and 7.04% of 181,303 project-event rows; deduplicating repeated project membership gives a 6.50% strict-gross sensitivity. The gross share is about 2.5–2.6% in the two ActPlane vendor strata, 9.7% for AgentSight Codex, and 21.9% in the AgentSkill paper case. The 61.6% value for the Skill-development case is a classification-boundary warning because Skill files are project products; excluding those in-repository product files lowers its adjusted share to 20.0%.

Strict same-target failure chains are rare. Sixteen chains contain 58 member calls, or 0.0320% of the 181,303 project-event rows; 12 chains are repeated edits, three are process polling, and one is a remote lookup. Thirteen of the 16 eventually recover on the exact target, while interleaved diagnostic failure clusters remain a separate sensitivity. This narrow result does not measure all diagnosis or recovery cost.

RQ7: Tool-Call Workload and Optimization Bounds

We position these results as descriptive workload characterization over the same persistent-workspace corpus. They jointly expose exact calls, workspace effects, recovery, and conservative system-opportunity bounds; they do not claim that tool distributions, retries, prefetch, speculation, caching, or concurrency are new mechanisms, nor do they report implemented speedups.

Composition, repetition, and timing observations. Shell tools account for 124,342/181,303 calls (68.6%); search/text commands make up 50.1% of the shell stratum, while native read/edit/write tools still account for 15.2% of all calls. Of 43,878 artifact-identity reads, 46.7% repeat an identity within the same prompt and source stream, and 76.2% of those repeats have no observed intervening mutation. Exact shell-command reruns account for 15.4% of shell calls, but only 0.6% are immediate. These are repetition indicators, not estimates of waste, because re-grounding, periodic status checks, and unobserved external changes remain possible. Across 8,410 timing-complete episodes, tool-busy interval union covers 25.17% of cumulative episode span and between-tool gaps cover 74.83%. Capping each gap at one hour or five minutes lowers the gap share to 69.57% or 65.54%. These episode-level observation bounds can overlap across concurrent lanes and do not estimate model computation or recoverable latency.

Existing and remaining concurrency. Native batch reconstruction places 33.2% of calls in multi-call batches. Among 172,346 adjacent episode edges in the deduplicated profile, 42,679 (24.76%) already start before the preceding result returns and another 1,801 (1.05%) are disjoint local reads already in one batch. Only 5,132 edges (2.98%) remain sequential, disjoint local-read candidates, while 49.47% of edges have unknown semantic dependence. The 42,679 native overlaps consume 86.0% of the 49,612 logical-parallel edges; this is an observation bound, not realized or remaining speedup. Thus existing concurrency is evidence of instruction-level parallelism, not unclaimed acceleration headroom.

Prediction, speculation, and event-driven opportunity bounds. A chronological actionable next-read predictor issued 2,911 prefetches and hit the exact next path 633 times, for 21.75% precision and 78.26% wasted prefetches; it did not predict range or formatting arguments. For 2,738 mutation-to-validation cycles, a last-test-command predictor matched 718 (26.22%) and could hide at most 1,564.511 seconds of observed test runtime. Running the eventual test after every edit would instead launch 11,206 stale versions among 13,944 edit-triggered executions, an 80.36% waste scenario. Wait-family tools account for 19,920 calls and 64.0% of summed per-call tool runtime, but most of that duration is external waiting rather than computation. After requiring the same tool and handle, passive calls, consecutive no-progress

results, and retaining one await per burst, an event-driven future could remove at most 1,456 calls/roundtrips, or 0.81% of all calls. This precision-oriented upper bound removes model wakeups and empty envelopes; it does not shorten the underlying process or subagent runtime. These opportunity bounds motivate versioned, debounced admission rather than eager execution; none is an implemented speedup or causal estimate.

Separate Tool Question: Measurement Capability

We freeze 12 complete Claude, Codex, or Gemini session files per project and derive 120 questions: 30 each about action-only, artifact-linked, cross-session, and final-state facts. A standalone checker imports neither the projection code nor `agent-session`; it reparses the 72 native files, reconstructs 1,721 artifact edges, independently verifies archived cutoff state, and reproduces every answer. We then compare Final State, Counts, pinned ProcGrep, and the artifact trajectory on the common denominator.

Figure 13 reports the deterministic matrix. As frozen on 2026-07-23, Final State answers all 30 final-state questions, and ProcGrep and Trajectory return identical answers on all 30 action questions while agreeing with the experiment's different source-direct action grammar on 18. This does not indicate failed ProcGrep preservation: the official adapter deliberately leaves some terminal reads unclassified. The decisive result is artifact-linked plus cross-session correctness. The frozen trajectory answers all 60 questions but returns only 32 correct and 28 wrong, with conditional accuracy from 0.0 to 1.0 across the six projects.

A follow-up audit classified the 28 errors: 14 were deliberately broader shell/scope evidence the frozen oracle excludes, and 14 were genuine bugs (native-root session joins, dropped failed-call effects, shell path extraction). Those classes plus one event-workdir bug were repaired, and the oracle itself was corrected (v4; 24 of 120 expected answers changed, each justified from source rows). The current revision answers all 60 artifact-linked and cross-session questions correctly against the corrected oracle (B 30/30, C 30/30, D 30/30) and 12/30 action-only questions; the A-family gap is the deliberate grammar difference above, since the trajectory preserves ProcGrep's action spine while the corrected oracle re-derives atom counts source-direct. We report this as repair-corpus conformance on the six-project benchmark, not as a general exact-fact capability claim.

In the fixed-reader, fixed-corpus comparison, the bounded Raw-log reader obtains 191/360 (53.1%) overall exact coverage and 94/180 (52.2%) B+C coverage. It produces 260/360 (72.2%) scoreable rows, with 191/260 (73.5%) exact accuracy among scoreable rows. 5/18 cells trigger the preregistered 1 MiB tool-return limit. The resulting comparison is mixed/inconclusive and supports no Trajectory superiority, necessity, speed, or cost claim. State Diff, Session Local, and OCPM Features remain N/A on this matrix.

Root-Disjoint Held-Out Conformance

The preregistered held-out corpus contains 70 native-session roots and 116 new question instances: 29 each in A, B, C, and D. The roots are disjoint from the repair corpus by file hash, native-root identity, and native-root/call identity; the questions have no exact-instance or full-row overlap with the earlier 120. Hamilton allocation assigns five questions per family to each project except `bpf-developer-tutorial`, which has four because only ten roots are eligible. The trajectory scores A 13/29, B 25/29, C 29/29, and D 29/29, with no abstentions. Thus the preregistered B+C decision is 54/58, a valid negative held-out result. Table 3 reports all 116 decisions; A is non-gating and retains the deliberate source-direct grammar difference.

The full attempted-edge ledger has 1,999 oracle rows and 2,017 projection rows: 1,998 match, one is missing, and 19 are extra. The confirmed-effect ledger has 1,845 oracle rows and 1,862 projection rows: 1,844 match, one is missing, and 18 are extra. Table 4 gives the registered metrics. The corrected session-order aggregate deduplicates production calls on $(\textit{native-session-id}, \textit{session-ordinal})$; all 70 unique pairs match. This reporting-layer correction does not change the negative decision, because B+C and both edge ledgers fail independently.

All 1,865 edge-call statuses match. The four B errors are confined to two identities: one additional failed read in `bpf-developer-tutorial` changes B1/B2 from 29/9 to 30/10, and one additional wrapped read in `eunomia.dev` changes B1/B2 from 44/21 to 45/22. The 20 row-level edge differences arise from seven native calls involving multi-source copy, recursive `git rm`, process substitution, a shell-rejected leading backslash, and static or custom `exec` wrappers. This post-run localization identifies a compound shell/wrapper path-admission boundary; it does not revise the frozen oracle or scores.

The earlier 60/60 and the held-out 54/58 have distinct source files, question instances, and denominators. The former remains a repair-corpus regression after root-cause repair; it is not pooled, rescaled, or denominator-matched with the held-out result. Exact C, D, call-status, and session-order results likewise do not erase the B and edge-ledger failures. We therefore make no general exact-conformance claim.

Projection Robustness Spot-Checks

The encoded-Claude-root spot-check covered the one dotted project and all 98 candidate transcript files, found no cwd-less or filter-reliant file and no missing pre-cutoff Tool call, and therefore reports `corpus_impact=false`. The `plausible_path_token` spot-check traced all 652 unique high-confidence rejected event operands to source edges and found zero unmatched operands and zero dropped file-action edges, because this filter gates event path groups rather than the shell-derived file edges.

Threats to Validity

Native Agent records can omit implicit subprocess effects, external editor changes, or actions performed outside instrumented tools. The study reports adapter and field coverage, and it does not convert missing effects into no-effect observations. Optional system-level AgentSight evidence is a future coverage extension, not a second logic layer in the present analysis.

Project	Family	Item-level oracle/trajectory pairs
ActPlane	A	A1 354/375*, A2 303/303, A3 70/46*, A4 5/5, A5 3/2*
ActPlane	B	B1 119/119, B2 42/42, B3 77/77, B4 read/read, B5 2/2
ActPlane	C	C1 4/4, C2 10/10, C3 1/1, C4 2/2, C5 26/26
ActPlane	D	D1 untracked/untracked, D2 untracked/untracked, D3 untracked/untracked, D4 untracked/untracked, D5 absent/absent
agentsight	A	A1 39/40*, A2 8/8, A3 9/3*, A4 1/1, A5 0/0
agentsight	B	B1 3/3, B2 1/1, B3 2/2, B4 read/read, B5 1/1
agentsight	C	C1 0/0, C2 0/0, C3 0/0, C4 0/0, C5 0/0
agentsight	D	D1 tracked/tracked, D2 tracked/tracked, D3 tracked/tracked, D4 tracked/tracked, D5 tracked/tracked
bpf-benchmark	A	A1 470/479*, A2 168/168, A3 30/7*, A4 9/9, A5 4/3*
bpf-benchmark	B	B1 96/96, B2 20/20, B3 76/76, B4 read/read, B5 3/3
bpf-benchmark	C	C1 6/6, C2 8/8, C3 0/0, C4 2/2, C5 30/30
bpf-benchmark	D	D1 tracked/tracked, D2 absent/absent, D3 tracked/tracked, D4 untracked/untracked, D5 absent/absent
bpf-dev-tutorial	A	A1 75/75, A2 73/73, A3 11/0*, A4 8/8
bpf-dev-tutorial	B	B1 29/30*, B2 9/10*, B3 20/20, B4 read/read
bpf-dev-tutorial	C	C1 3/3, C2 5/5, C3 1/1, C4 4/4
bpf-dev-tutorial	D	D1 tracked/tracked, D2 tracked/tracked, D3 tracked/tracked, D4 tracked/tracked
bpftime	A	A1 163/182*, A2 18/18, A3 52/8*, A4 3/3, A5 2/2
bpftime	B	B1 10/10, B2 8/8, B3 2/2, B4 read/read, B5 2/2
bpftime	C	C1 1/1, C2 5/5, C3 1/1, C4 11/11, C5 7/7
bpftime	D	D1 tracked/tracked, D2 absent/absent, D3 tracked/tracked, D4 tracked/tracked, D5 tracked/tracked
eunomia.dev	A	A1 176/196*, A2 98/48*, A3 78/25*, A4 4/3*, A5 2/1*
eunomia.dev	B	B1 44/45*, B2 21/22*, B3 23/23, B4 read/read, B5 2/2
eunomia.dev	C	C1 7/7, C2 9/9, C3 0/0, C4 1/1, C5 23/23
eunomia.dev	D	D1 tracked/tracked, D2 tracked/tracked, D3 tracked/tracked, D4 tracked/tracked, D5 untracked/untracked

Table 3: All 116 root-disjoint held-out question decisions. Each item is shown as oracle/trajectory; * marks a wrong answer. All rows are answered, with no abstentions. The B+C gate contains four wrong B rows and no wrong C rows.

Ledger	Precision	Recall	F1
Attempted edges	0.9906	0.9995	0.9950
Confirmed-effect edges	0.9903	0.9995	0.9949
Edge-call statuses	1.0000	1.0000	1.0000
Session order	1.0000	1.0000	1.0000

Table 4: Root-disjoint held-out ledger conformance.

The source-direct tool experiment also found material disagreement between the frozen projection and native records: 28/60 answered artifact-linked or cross-session facts were wrong as of 2026-07-23. The completed error taxonomy separated deliberately broader shell/scope admission (14 rows) from genuine path, artifact-identity, native-root join, and event-workdir errors (14 rows); the genuine classes were repaired and the oracle corrected. The current revision matches the corrected repair-corpus oracle on all 60 B+C questions, and RQ1–RQ4 were recomputed at that revision. A separate preregistered root-disjoint corpus scores 54/58 B+C and exposes residual compound shell/wrapper path-admission differences; it is neither combined with nor overridden by the 60/60 regression. The action-only family still differs deliberately, because the trajectory preserves ProcGrep's action spine while the corrected oracle re-derives atom counts source-direct. RQ5 is better protected by its separate 2,063-stream checker, and RQ6 by an independent public-data reconstruction. These results do not turn N/A into observed absence and do not support general exact conformance or a population-level capability claim.

Final workspace state cannot prove content-level survival of every historical edit. We therefore separate file existence, Git-tracked state, and independently supported content survival. Successful validation is association rather than coverage or correctness evidence.

The six cases are selected from local author-associated projects. They provide deep, source-native longitudinal evidence but cannot establish population-wide rates. The public RQ6 samples independently reproduce only compatible within-attempt path-locality and return relations. Open-SWE-Traces consists of synthetic benchmark attempts, and IdeaTrail is reverse-synthesized under one workflow; neither validates natural cross-session persistence, re-grounding, or Skill effects.

Skill, harness, model, task, and project often change together. Source-native attribution makes Skill-conditioned activity measurable, but it does not identify positive and negative configuration exposure or remove task/phase confounding. RQ5 therefore reports source-attributed composition and a negative repeatability result rather than an observational outcome or causal effect. Likewise, repeated actions, unvisited artifacts, and long validation gaps are process facts rather than automatic failure labels.

Related Work

Agent trajectory studies. Software-engineering trajectory studies analyze action sequences, reasoning, testing, and strategy (Bouzenia and Pradel 2025; Mehtiyev and Assunção 2026). TRAJEVAL identifies coherence collapse after reaching useful intermediate code (Kim et al. 2026). ProcGrep

induces canonical action procedures and behavioral fingerprints (Oderinwale 2026). We reuse these action-level precedents and focus the novelty question on persistent artifact identity and cross-session lineage.

Long-horizon and persistent work. AgingBench studies reliability across many sessions (Zhu et al. 2026), while context-management work shows that plans can lose influence (Mehta and Datta 2026). OR-Space evaluates persistent multi-artifact workspaces (Zhou et al. 2026). These works motivate the setting but do not measure introduced-artifact persistence, reuse, validation association, and re-grounding across real project histories.

Diagnosis and harness analysis. AgentDiagnose and AgentRx diagnose completed trajectories (Ou et al. 2025; Barke et al. 2026); TrajAudit retrieves evidence from repository traces (Wang, Xie, and Huo 2026); and HarnessFix links failed trajectories to harness mechanisms (Chen et al. 2026). We do not claim a new diagnostic labeler or causal harness repair. Our tool evaluation asks only which process facts are measurable from source-linked artifact lineage.

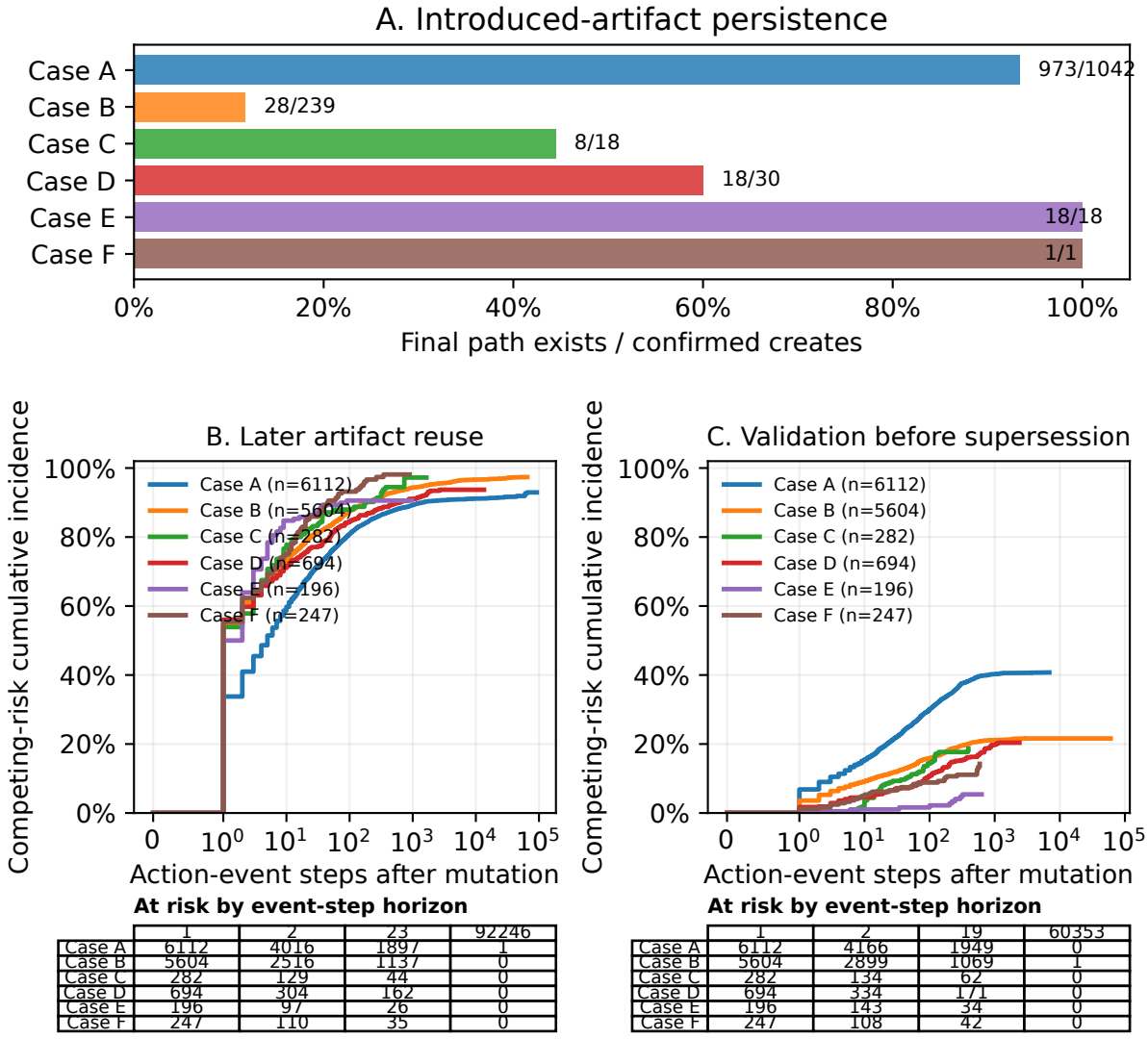
Ethical Considerations

Native trajectories can contain private prompts, source code, paths, secrets, and experimental data. Local analysis does not imply permission to publish raw sessions. Released evidence must follow repository and data permissions, remove credentials and personal content, and prefer aggregate or explicitly approved source-linked examples.

Process measurements can be misused as productivity scores. We therefore do not rank people, treat activity volume as progress, or label an observed pattern as waste without independent task evidence. The intended consumer is an Agent or automatic diagnostic process studying its own authorized workspace.

Conclusion

Long-running Agent work persists in artifacts while session context disappears. Studying only final state, commits, or action volume therefore misses the process users need to understand: which work persists, is revisited, receives recognized validation, accumulates repeated mutation, and migrates across a workspace. Across six projects, later reuse is consistently high but weakly ordered by action volume, while repeated mutation is both common and strongly concentrated. Validation, session-continuity, and skill analyses also expose hard source-coverage boundaries rather than portable success, reset, or causal harness effects. The workload study also finds that native overlap consumes most explicit logical parallelism, leaving only conservative opportunity bounds. Our workspace-centered analysis makes these process facts inspectable while keeping unsupported comparisons explicitly N/A. Source-direct conformance moves from 32/60 to 60/60 on the corrected repair corpus, while a separate preregistered root-disjoint test obtains 54/58 and localizes residual mismatch to compound shell/wrapper paths. The resulting research boundary is concrete—persistent workspace evolution remains the useful unit, but neither corpus supports general exact conformance or broader population-level and tool-capability claims.



At risk by event-step horizon

	1	2	23	92246
Case A	6112	4016	1897	1
Case B	5604	2516	1137	0
Case C	282	129	44	0
Case D	694	304	162	0
Case E	196	97	26	0
Case F	247	110	35	0

At risk by event-step horizon

	1	2	19	60353
Case A	6112	4166	1949	0
Case B	5604	2899	1069	1
Case C	282	134	62	0
Case D	694	334	171	0
Case E	196	143	34	0
Case F	247	108	42	0

Figure 3: RQ1's three observable progress dimensions. Panel A reports exact final-path persistence only for adapter-recognized confirmed creates. Panels B and C use Aalen–Johansen cumulative incidence over native Agent action-event distance: deletion competes with later reuse, while the next same-artifact mutation/delete competes with recognized validation. N/A is source ineligibility, not zero progress. After projection repair all 6/6 cases expose an eligible create and a recognized success (3/6 before repair).

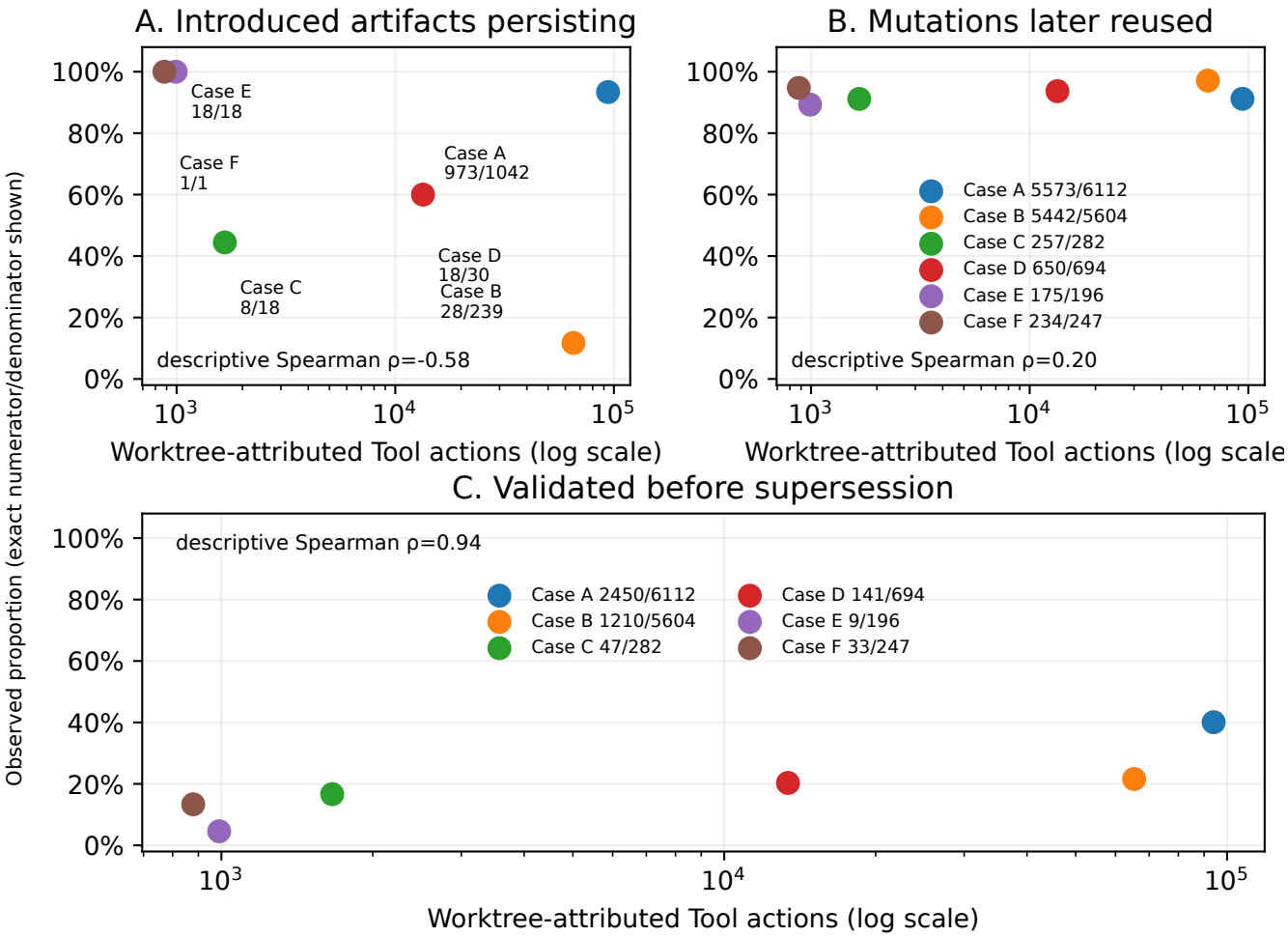


Figure 4: Worktree-attributed Tool action volume versus exact observed RQ1 proportions. Labels show numerator/denominator rather than fitted values. All three dimensions cover six cases after projection repair; each panel reports only a descriptive rank correlation, and actions are not treated as independent cases.

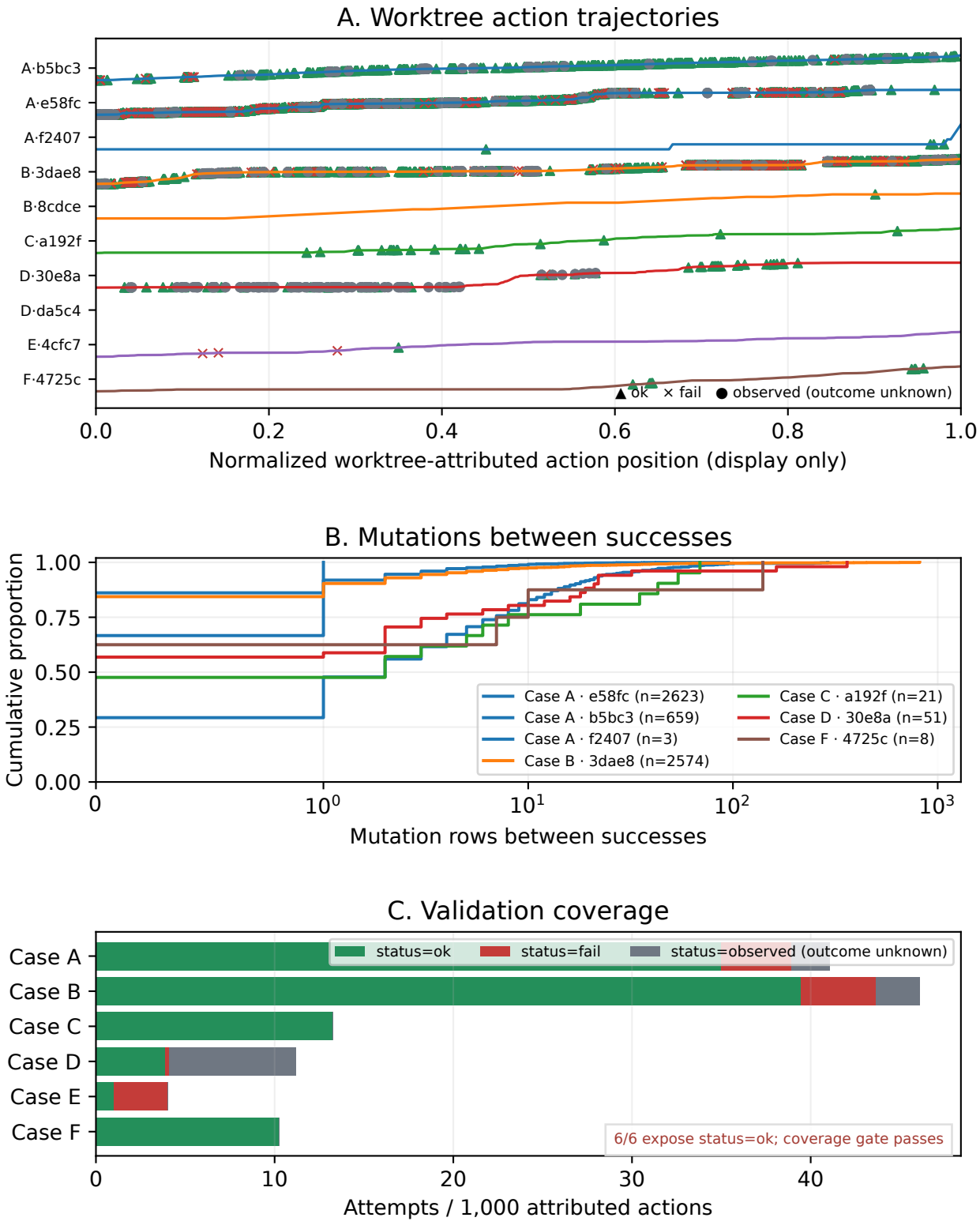
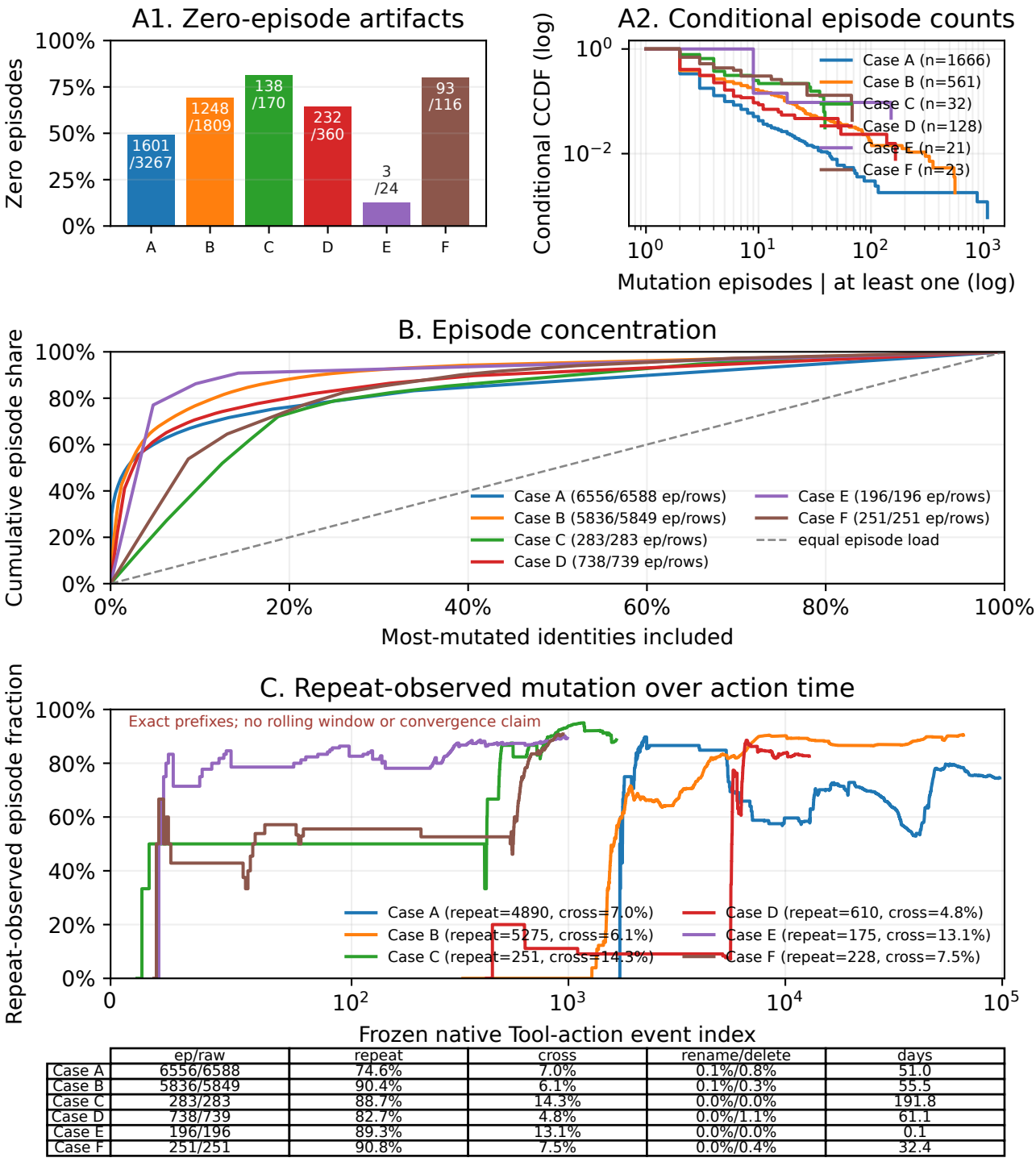
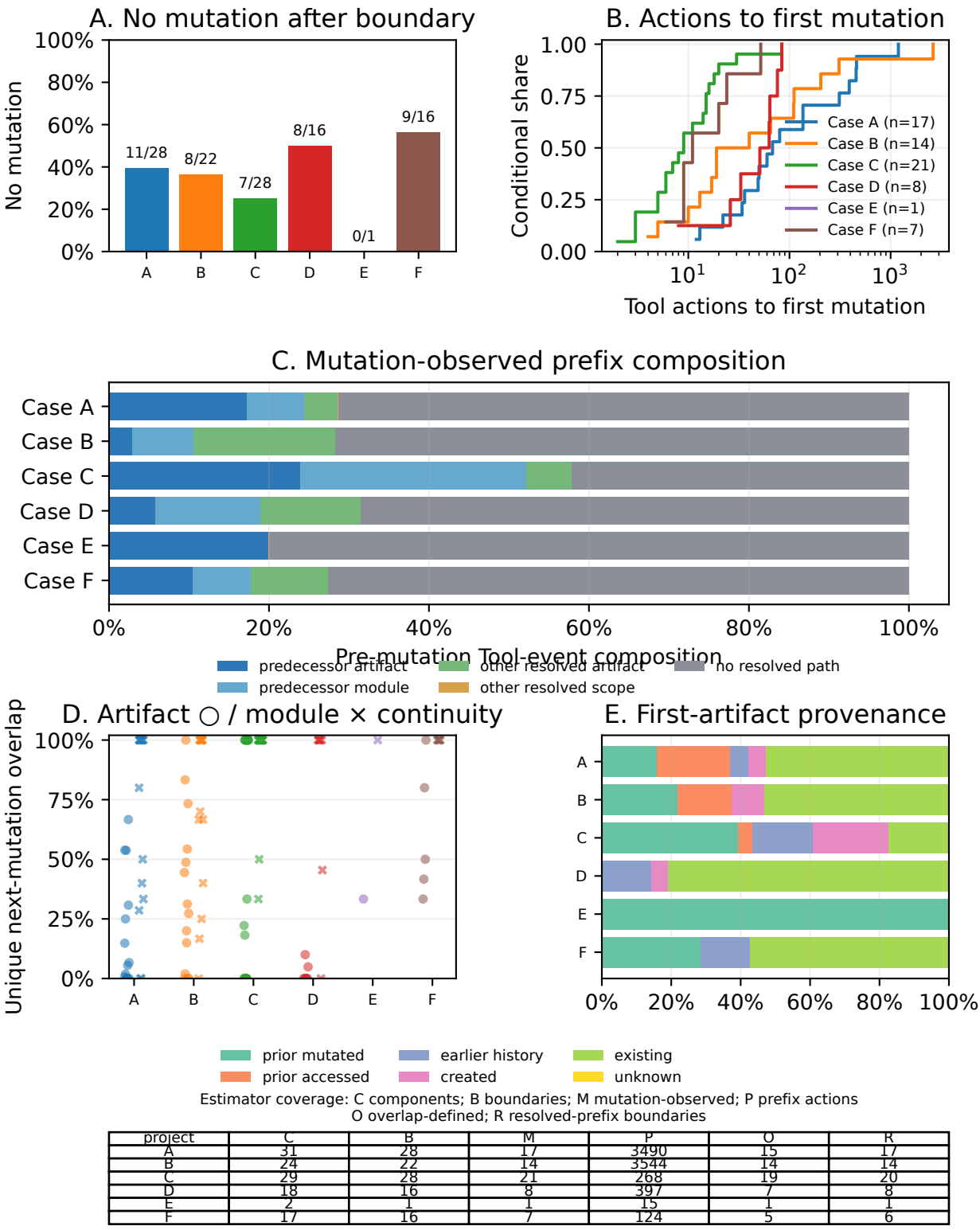


Figure 5: RQ2 recognized-validation dynamics. Panel A follows native Agent action order in every worktree lane; mutation effects project to the affected worktree while validation remains on the command worktree. Panel B reports complete worktree-local intervals between recognized successful validations. Panel C exposes project-level outcome coverage. All 6/6 cases contain a recognized success after projection repair (3/6 before repair).



	ep/raw	repeat	cross	rename/delete	days
Case A	6556/6588	74.6%	7.0%	0.1%/0.8%	51.0
Case B	5836/5849	90.4%	6.1%	0.1%/0.3%	55.5
Case C	283/283	88.7%	14.3%	0.0%/0.0%	191.8
Case D	738/739	82.7%	4.8%	0.0%/1.1%	61.1
Case E	196/196	89.3%	13.1%	0.0%/0.0%	0.1
Case F	251/251	90.8%	7.5%	0.0%/0.4%	32.4

Figure 6: RQ1 repeated-mutation structure for 5,746 observed identities, 2,431 mutated identities, and 13,860 mutation episodes (13,906 source rows). The all-identity zero mass is separated from the conditional CCDF; the concentration curve uses the exact fractional 10% boundary; and the repeat fraction evolves as an action-atomic post-action step function.



Estimator coverage: C components; B boundaries; M mutation-observed; P prefix actions
O overlap-defined; R resolved-prefix boundaries

project	C	B	M	P	O	R
A	31	28	17	3490	15	17
B	24	22	14	3544	14	14
C	29	28	21	268	19	20
D	18	16	8	397	7	8
E	2	1	1	15	1	1
F	17	16	7	124	5	6

All four-project gates stopped: component continuity only; not a reset, resume, memory, or forgetting estimate.

Figure 7: RQ4 source-session component continuity. Adjacent, non-overlapping concurrency components avoid inventing a serial order among overlapping sessions. Timing is conditional on an observed first mutation; empty denominators remain undefined. The stop notice makes the failed four-project gate explicit, so all panels remain within-case descriptions.

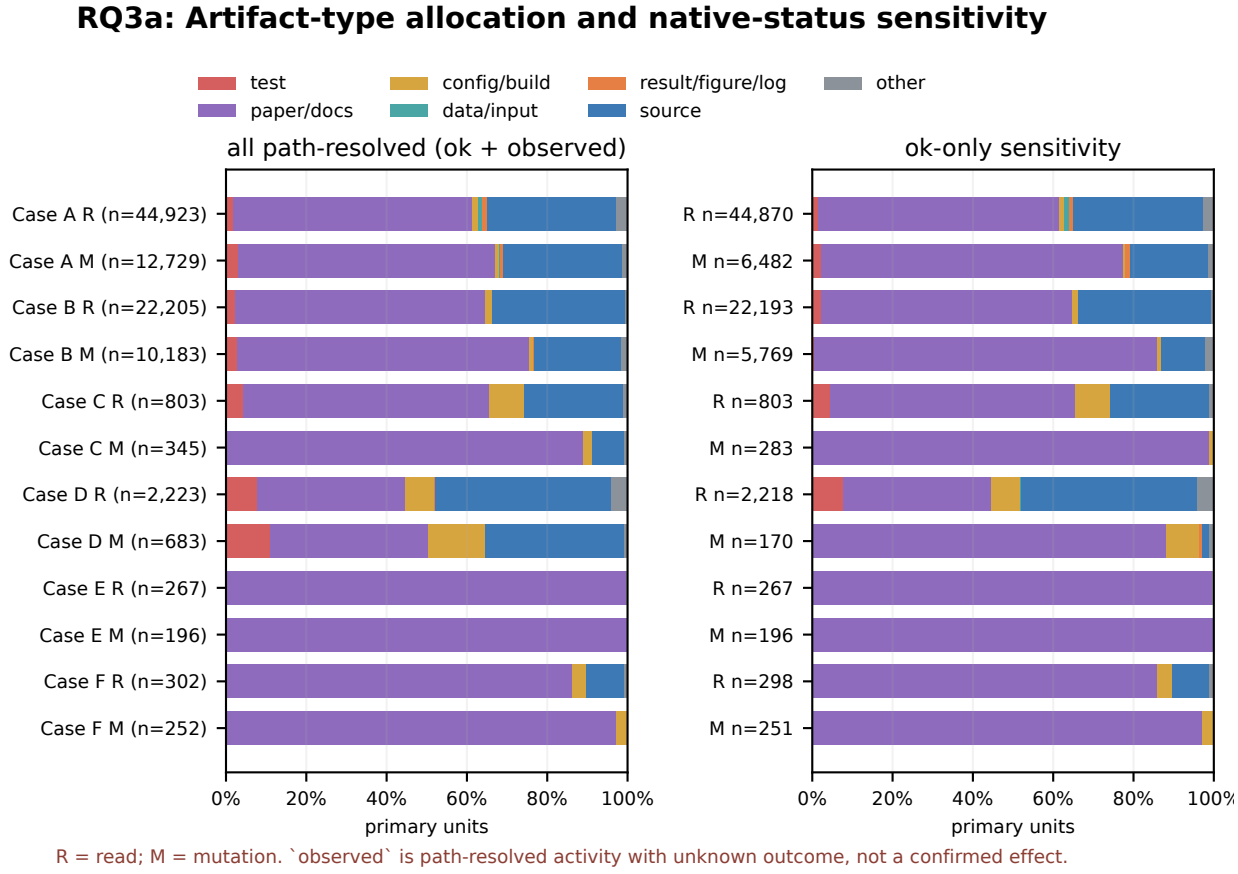


Figure 8: RQ3 path-resolved workspace activity allocation by artifact class, action effect, and source status. One multi-path Tool call contributes fractionally across its resolved paths in the call-weighted view; exact action-weighted rows are retained separately.

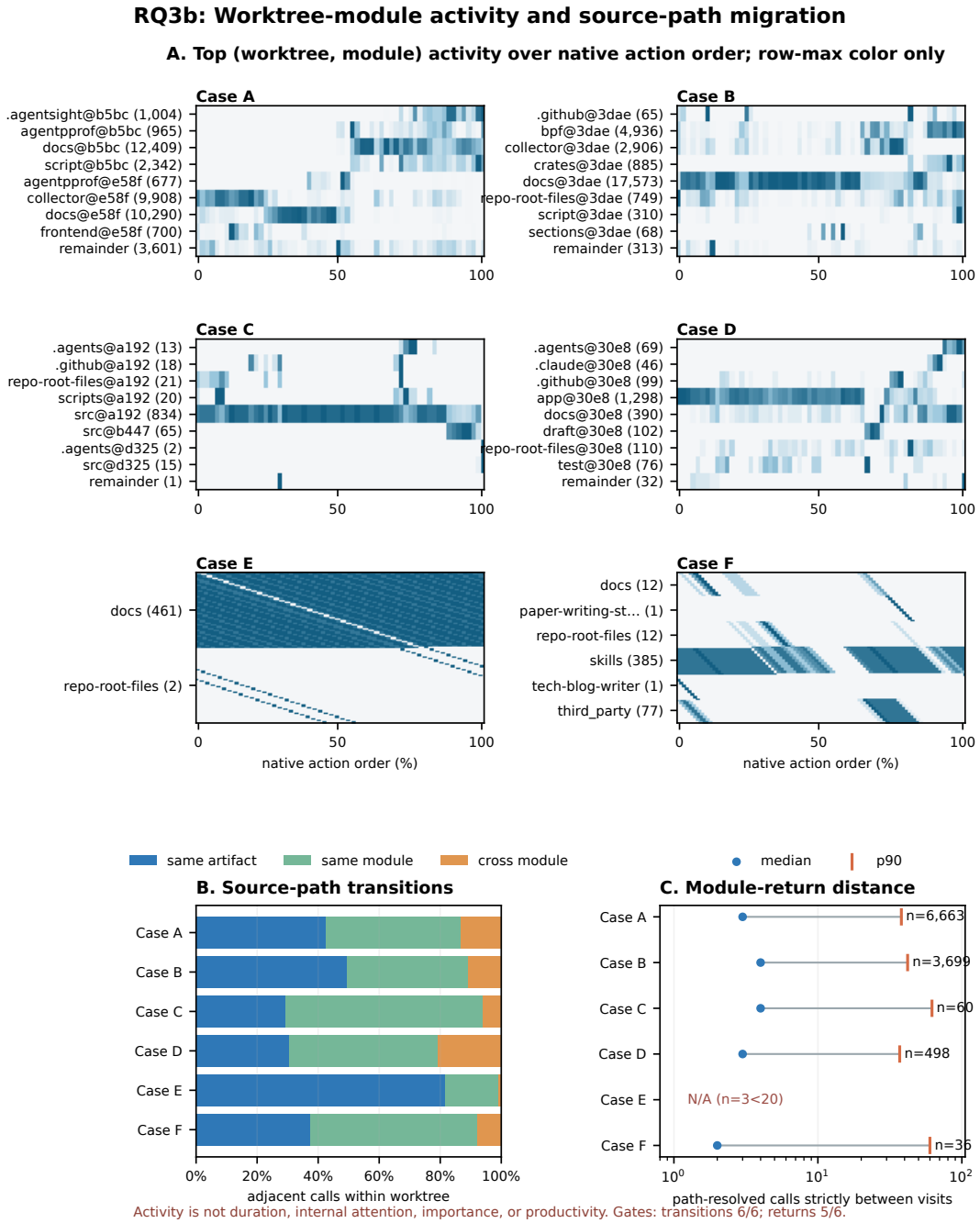


Figure 9: RQ3 workspace focus migration in native call order. Transitions are computed within worktree lanes as same artifact, same module, or cross module; return gaps count intervening calls. Module names and worktree-module keys are reported separately, and low-support estimates remain N/A.

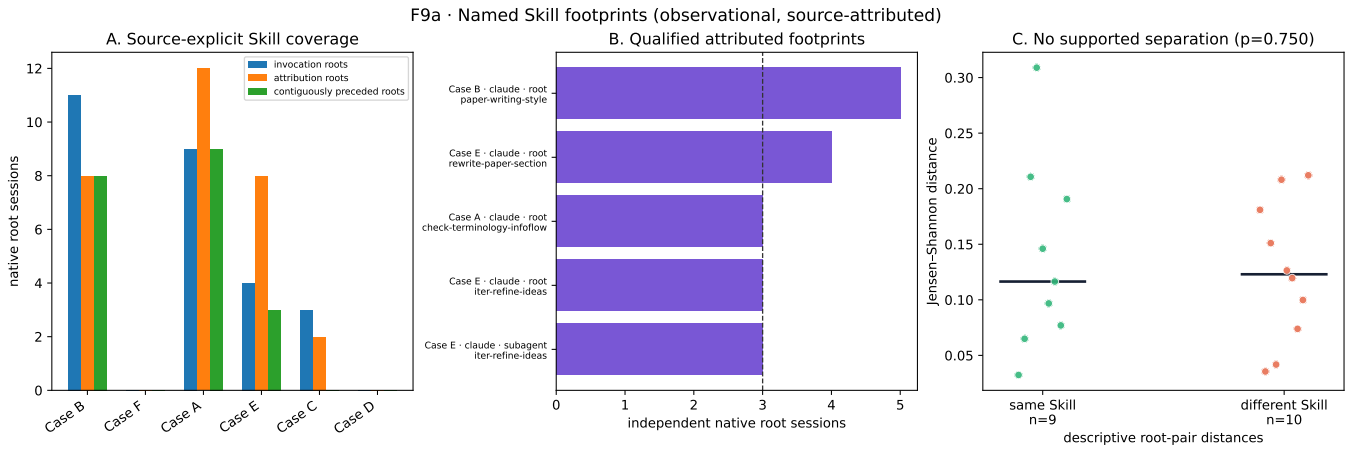


Figure 10: RQ5 named-Skill evidence. Transcript files sharing a native root session are one independent block. The support gate requires at least three roots. Same-Skill and different-Skill distances are observational matched comparisons, not causal harness effects.

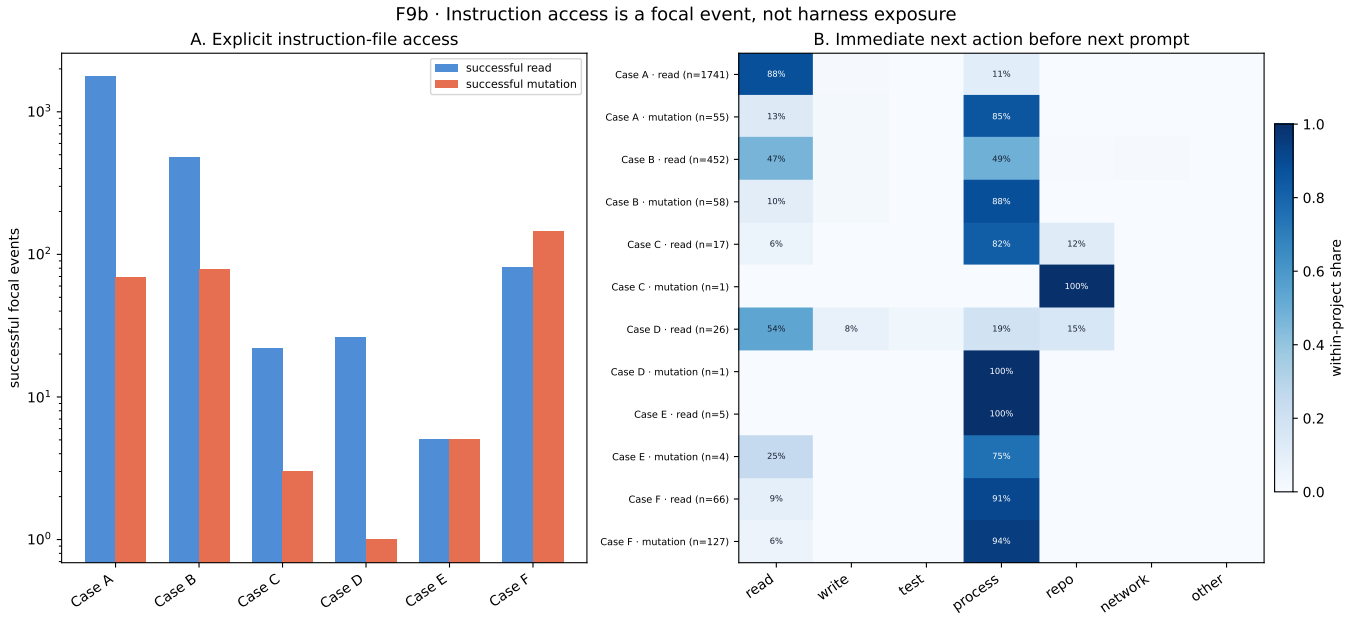


Figure 11: RQ5 instruction-file focal events and immediate following activity. Reads and mutations remain separate, each focal event contributes at most one following Tool action, and next-action shares are normalized within each project rather than pooled across the six selected cases.

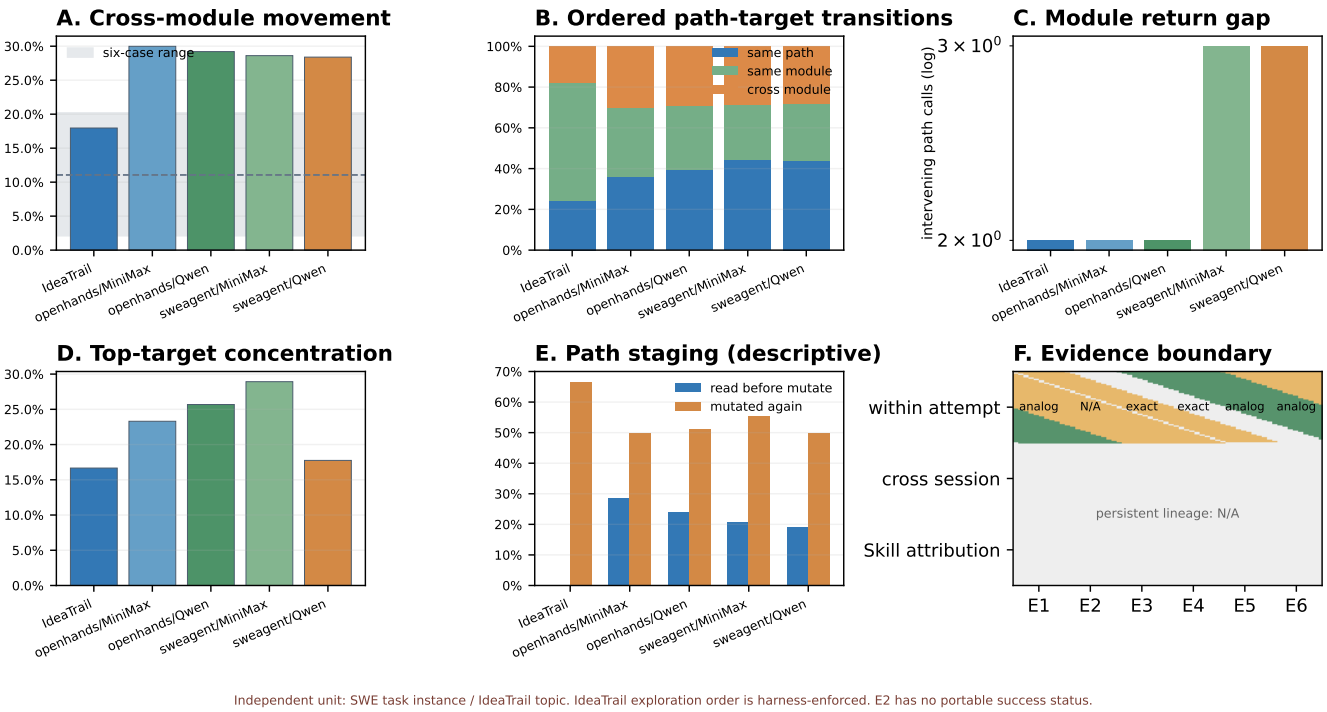


Figure 12: RQ6 external boundary over 64 independent tasks per Open-SWE stratum (256 selections, 255 unique task IDs across strata) and 64 IdeaTrail topics. E3/E4 share the local path-target/module-return projection; E1/E5/E6 are descriptive or analogous, and E2 is N/A. Public traces do not expose persistent cross-session artifact lineage or exact Skill attribution.

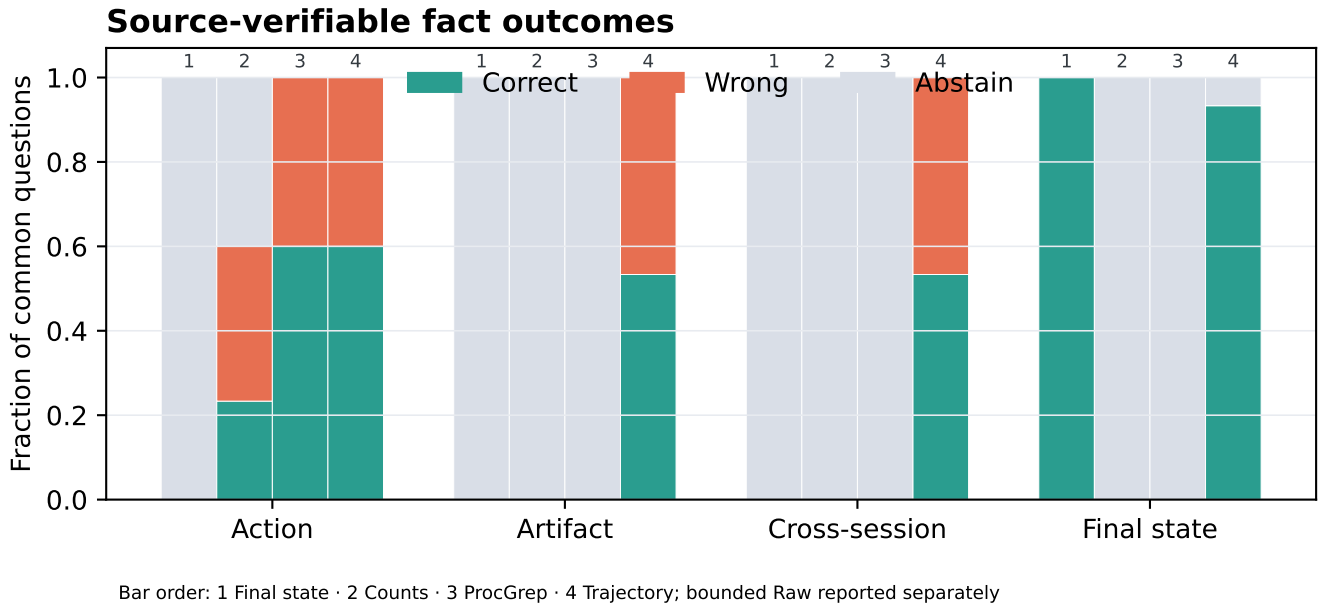


Figure 13: Source-direct measurement conformance over 120 questions for the repaired implementation against the corrected v4 oracle. Each bar partitions the common 30-question family into correct, wrong, and abstain. The frozen implementation scored 32/60 on artifact plus cross-session families (2026-07-23); after root-cause repair they are 60/60. Bounded Raw is evaluated separately under the registered fixed-reader protocol.

References

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*.

Chen, M.; Wang, J.; Liu, Z.; Wang, Y.; Zheng, H.; and Wang, Q. 2026. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324.

Kim, M.; Wang, D.; Cui, S.; Farmahinifarahani, F.; Zhuo, T. Y.; Garg, S.; Ray, B.; Mukherjee, R.; and Kumar, V. 2026. Coherence Collapse: Diagnosing Why Code Agents Fail After Reaching the Right Code. arXiv:2603.24631.

Mehta, A.; and Datta, A. 2026. Plans Don't Persist: Why Context Management Is Load Bearing for LLM Agents. arXiv:2606.22953.

Mehtiyev, T.; and Assunção, W. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. arXiv:2604.02547.

Oderinwale, H. 2026. Agent Trajectories as Programs: Fingerprinting and Programming Coding-Agent Behavior. arXiv:2606.16988.

Ou, T.; Guo, W.; Gandhi, A.; Neubig, G.; and Yue, X. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 207–215.

Wang, M.; Xie, X.; and Huo, Y. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563.

Zhou, C.; Lu, X.; Zhao, J.; Lin, J.; Ge, D.; and Ye, Y. 2026. ORSpace: A Full-Lifecycle Workspace Benchmark for Industrial Optimization Agents. arXiv:2605.28158.

Zhu, J.; Ro, Y.; Robertson, J. T.; Wang, K.; Li, J.; Vikalo, H.; Akella, A.; and Wang, Z. 2026. Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems. arXiv:2605.26302.